

Consultas SQL en PostgreSQL: el mapa para armar cualquier consulta

Diego Muñoz

16 Junio 2026

La receta para responder cualquier pregunta con SQL

- Conocimientos previos: crear tablas, insertar datos y hacer SELECT básicos con JOIN.
- Objetivo: **armar cualquier consulta**, por compleja que parezca.
- La idea central: toda consulta sigue **el mismo esqueleto**. Si entiendes el esqueleto, solo vas rellenando piezas.

El esqueleto de toda consulta

SELECT	columnas / expresiones	-- <i>qué quiero ver</i>
FROM	tabla	-- <i>de dónde</i>
JOIN	otra_tabla ON condicion	-- <i>con qué la combino</i>
WHERE	condicion_de_filas	-- <i>qué filas dejo pasar</i>
GROUP BY	columnas	-- <i>cómo agrupo</i>
HAVING	condicion_de_grupos	-- <i>qué grupos dejo pasar</i>
ORDER BY	columnas	-- <i>cómo ordeno</i>
LIMIT	n OFFSET m;	-- <i>cuántas filas</i>

- Las únicas cláusulas obligatorias en cada consulta son **SELECT** y **FROM**.
- El resto se agrega **solo cuando se necesita**.

El orden en que se ESCRIBE no es el orden en que se EJECUTA

- Se escribe empezando por SELECT, pero la base de datos **no** lo procesa en ese orden.

Orden lógico de ejecución

1. FROM / JOIN → arma el conjunto de filas combinando tablas.
2. WHERE → descarta filas que no cumplen.
3. GROUP BY → agrupa las filas restantes.
4. HAVING → descarta grupos que no cumplen.
5. SELECT → calcula columnas y expresiones.
6. DISTINCT → elimina duplicados.
7. ORDER BY → ordena el resultado.
8. LIMIT / OFFSET → recorta cuántas filas se devuelven.

¿Por qué importa el orden de ejecución?

- El alias definido en SELECT se puede usar en GROUP BY y ORDER BY, pero **no** en WHERE ni en HAVING: ahí hay que repetir la expresión completa.
- Un agregado como AVG o COUNT **no** se filtra en WHERE: se calcula en el paso de agrupación, **después** del WHERE. Para filtrar por un agregado existe HAVING.

El alias del SELECT no existe en WHERE

-- ERROR: column "dif" does not exist

```
SELECT evaluacion, nota - 4.0 AS dif  
FROM nota WHERE dif > 0;
```

-- BIEN: repetir la expresión en el WHERE

```
SELECT evaluacion, nota - 4.0 AS dif  
FROM nota WHERE nota - 4.0 > 0;
```

- El WHERE corre antes que el SELECT; el alias dif aún no existe.

Un agregado no se filtra en WHERE

-- ERROR: aggregate functions are not allowed in WHERE

```
SELECT curso_id, AVG(nota)
FROM nota WHERE AVG(nota) > 5
GROUP BY curso_id;
```

-- BIEN: filtrar el agregado con HAVING

```
SELECT curso_id, AVG(nota)
FROM nota
GROUP BY curso_id HAVING AVG(nota) > 5;
```

- El agregado se calcula después del WHERE; para filtrarlo está HAVING.

El dataset de la clase

Reutilizamos el modelo de cursos de la clase de SQL intro:

- `alumno(alumno_id, nombre)`
- `profesor(profesor_id, nombre)`
- `curso(curso_id, nombre, profesor_id)`
- `alumno_curso(alumno_id, curso_id) → qué alumno cursa qué`
- `horario(horario_id, dia, hora_inicio, hora_fin, sala_id, curso_id)`
- `sala(sala_id, nombre, capacidad)`

Una tabla nueva para esta clase: nota

```
CREATE TABLE nota (  
    nota_id SERIAL PRIMARY KEY,  
    alumno_id INTEGER NOT NULL REFERENCES alumno(alumno_id),  
    curso_id INTEGER NOT NULL REFERENCES curso(curso_id),  
    evaluacion TEXT NOT NULL,          -- 'Prueba 1', 'Prueba 2', 'Examen'  
    nota DECIMAL(3,1) NOT NULL CHECK (nota BETWEEN 1.0 AND 7.0),  
    fecha DATE NOT NULL  
);
```

- Cada inscripción tiene varias evaluaciones con su nota y fecha.
- Nos da **números y fechas** para practicar agregados y funciones ventana.

Cómo cargar los datos como migraciones

- En la carpeta `model/`, tres migraciones numeradas:
 - `000-init-schema.sql`: crea el schema + `schema_migrations` + modelo base.
 - `001-add-table-notas.sql`: crea la tabla `nota`.
 - `002-insert-values.sql`: inserta todos los datos.
- Se aplican **en orden**:

```
psql -d tu_bd -f model/000-init-schema.sql
```

```
psql -d tu_bd -f model/001-add-table-notas.sql
```

```
psql -d tu_bd -f model/002-insert-values.sql
```

- Cada archivo se auto-registra en `schema_migrations`; revisa con `SELECT * FROM schema_migrations;`.

Parte 1: el esqueleto del SELECT

Empecemos por lo mínimo

- Toda consulta nace de **dos piezas**: qué quiero ver con SELECT y de dónde con FROM.
- Sobre esa base iremos agregando todo lo demás.

SELECT ... FROM: lo mínimo

-- Todas las columnas de todos los alumnos

```
SELECT * FROM alumno;
```

-- Solo las columnas que me interesan

```
SELECT nombre FROM alumno;
```

- * trae **todas** las columnas. Útil para explorar, pero en consultas reales conviene pedir solo lo necesario.

Proyección: expresiones y alias

-- Puedo calcular columnas, no solo leerlas

SELECT

evaluacion,

nota,

nota - 4.0 **AS** distancia_a_la_aprobacion

FROM nota;

- AS le pone un **alias** a la columna calculada.
- El alias es opcional: nota - 4.0 distancia también funciona, pero con AS se lee mejor.

DISTINCT: eliminar filas repetidas

-- ¿Qué nombres de evaluación existen?

```
SELECT DISTINCT evaluacion FROM nota;
```

-- DISTINCT aplica a la COMBINACIÓN de columnas

```
SELECT DISTINCT alumno_id, curso_id FROM nota;
```

- DISTINCT mira la fila completa que proyectaste: dos filas son “iguales” solo si **todas** sus columnas coinciden.

ORDER BY: ordenar el resultado

-- Notas de mayor a menor

```
SELECT evaluacion, nota  
FROM nota  
ORDER BY nota DESC;
```

-- Por varias columnas: primero por curso, luego por nota

```
SELECT curso_id, nota  
FROM nota  
ORDER BY curso_id ASC, nota DESC;
```

- ASC es ascendente y el valor por defecto; DESC es descendente.
- Se puede ordenar por columnas que **no** aparecen en el SELECT.

ORDER BY y los NULL

```
SELECT evaluacion, nota  
FROM nota  
ORDER BY nota DESC NULLS LAST;
```

- En PostgreSQL los NULL se ordenan **al final** en ASC y **al inicio** en DESC por defecto.
- NULLS FIRST / NULLS LAST te deja controlarlo explícitamente.

LIMIT y OFFSET: recortar y paginar

-- Las 5 mejores notas

```
SELECT evaluacion, nota  
FROM nota  
ORDER BY nota DESC  
LIMIT 5;
```

-- Página 2, de a 5 filas (salta las primeras 5)

```
SELECT evaluacion, nota  
FROM nota  
ORDER BY nota DESC  
LIMIT 5 OFFSET 5;
```

- LIMIT sin ORDER BY da resultados **impredecibles**: siempre ordena antes.

Parte 2: filtrar filas con WHERE

Quedarse solo con las filas que importan

- WHERE se evalúa **fila por fila**: la deja pasar si la condición es verdadera.
- Es el filtro más usado de SQL.

WHERE: comparadores básicos

```
SELECT evaluacion, nota
FROM nota
WHERE nota >= 4.0;           -- aprobados
```

- Comparadores:
 - = igual
 - != o <> distinto
 - < menor / > mayor
 - <= menor o igual / >= mayor o igual
- Funcionan con números, texto y fechas.

```
SELECT * FROM nota WHERE fecha > '2025-05-01';
```

Combinar condiciones: AND, OR, NOT

-- Aprobados en el examen

```
SELECT * FROM nota  
WHERE nota >= 4.0 AND evaluacion = 'Examen';
```

-- Notas extremas

```
SELECT * FROM nota  
WHERE nota < 3.0 OR nota > 6.5;
```

- **Cuidado con la precedencia:** AND se evalúa antes que OR.
- Ante la duda, **usa paréntesis:**

```
WHERE evaluacion = 'Examen' AND (nota < 3.0 OR nota > 6.5)
```

Operadores cómodos: BETWEEN, IN

-- Rango (inclusivo en ambos extremos)

```
SELECT * FROM nota
WHERE nota BETWEEN 4.0 AND 5.0;  -- = nota >= 4.0 AND nota <= 5.0
```

-- Pertenencia a un conjunto

```
SELECT * FROM nota
WHERE evaluacion IN ('Prueba 1', 'Prueba 2');
```

- BETWEEN a AND b equivale a $a \geq a$ AND $a \leq b$.
- IN (...) evita escribir muchos OR.

Buscar texto: LIKE e ILIKE

-- Nombres que empiezan con 'Prueba'

```
SELECT * FROM nota WHERE evaluacion LIKE 'Prueba%';
```

-- Alumnos cuyo nombre contiene 'mar' sin importar mayúsculas

```
SELECT * FROM alumno WHERE nombre ILIKE '%mar%';
```

- % = cualquier cantidad de caracteres; _ = exactamente uno.
- ILIKE es como LIKE pero **ignora mayúsculas/minúsculas**; es propio de PostgreSQL.

El gran tramposo: NULL

- NULL no es cero ni cadena vacía: significa “**valor desconocido**”.
- Por eso **no** se compara con =:

-- MAL: nunca devuelve filas

```
SELECT * FROM nota WHERE nota = NULL;
```

-- BIEN

```
SELECT * FROM nota WHERE nota IS NULL;
```

```
SELECT * FROM nota WHERE nota IS NOT NULL;
```

- Con NULL, una condición no es solo verdadera o falsa: puede ser **desconocida**, el valor UNKNOWN.
- WHERE solo deja pasar lo que es **verdadero**: descarta falso y desconocido.
- Consecuencia: si una columna tiene NULL, condiciones como `nota <> 7.0` **no** incluyen esa fila, aunque “intuitivamente” un desconocido no sea 7.

Parte 3: combinar tablas con JOIN

Los datos viven repartidos

- Una nota guarda `alumno_id` y `curso_id` como **números**, es decir ids.
- Para ver el nombre del alumno o del curso, hay que **unir** tablas.

Repaso: tipos de JOIN

- INNER JOIN: solo las filas con coincidencia en **ambas** tablas.
- LEFT JOIN: todas las de la izquierda más lo que calce a la derecha; lo que no calce queda en NULL.
- RIGHT JOIN: lo simétrico.
- FULL JOIN: todo de ambos lados.

```
SELECT n.nota, a.nombre  
FROM nota AS n  
INNER JOIN alumno AS a ON a.alumno_id = n.alumno_id;
```

Convención de nombres: <tabla>_id

Por qué la PK se llama curso_id y no id

- Cada tabla nombra su PK como <tabla>_id: alumno_id, curso_id, nota_id...
- La FK que apunta a una tabla **reusa ese mismo nombre**: nota.curso_id referencia a curso.curso_id.
- Como la columna de unión se llama **igual** en ambos lados, puedes unir con USING:

-- con esta convención

```
SELECT * FROM nota JOIN curso USING (curso_id);
```

-- sin ella (PK = id, FK = curso) no queda otra que ON

```
SELECT * FROM nota JOIN curso ON curso.id = nota.curso;
```

ON vs USING

```
-- El mismo JOIN, dos formas de escribirlo.  
-- ON: condición explícita, sirve siempre  
SELECT *  
FROM nota n  
JOIN curso c ON c.curso_id = n.curso_id;  
-- USING: atajo porque la columna se llama igual (curso_id)  
SELECT *  
FROM nota  
JOIN curso USING (curso_id);
```

- Las dos hacen **lo mismo**. USING funciona gracias a la convención de nombres y además **fusiona** la columna repetida: aparece una sola vez. Si los nombres difieren, usa ON.

Encadenar varios JOIN

-- Nota, alumno, curso y profesor en una sola consulta

```
SELECT a.nombre AS alumno,  
       c.nombre AS curso,  
       p.nombre AS profesor,  
       n.nota  
FROM nota n  
JOIN alumno a ON a.alumno_id = n.alumno_id  
JOIN curso c ON c.curso_id = n.curso_id  
JOIN profesor p ON p.profesor_id = c.profesor_id;
```

- Cada JOIN agrega una tabla más al conjunto.
- Los **alias cortos** como a, c, p hacen la consulta legible.

Self join: una tabla consigo misma

```
-- Pares de cursos dictados por el mismo profesor
SELECT c1.nombre AS curso_a,
       c2.nombre AS curso_b
FROM curso c1
JOIN curso c2
  ON c1.profesor_id = c2.profesor_id
 AND c1.curso_id < c2.curso_id;    -- evita repetir pares
```

- La misma tabla aparece **dos veces** con alias distintos.

Anti-join: lo que NO tiene coincidencia

```
-- Alumnos que NO tienen ninguna nota registrada
SELECT a.nombre
FROM alumno a
LEFT JOIN nota n ON n.alumno_id = a.alumno_id
WHERE n.nota_id IS NULL;
```

- Truco clásico: LEFT JOIN + WHERE ... IS NULL.
- Las filas sin pareja quedan con NULL en las columnas de la derecha; ese IS NULL las atrapa.

Semi-join: existe al menos una coincidencia

```
-- Alumnos que SÍ tienen alguna nota (sin duplicar al alumno)
SELECT a.nombre
FROM alumno a
WHERE EXISTS (
    SELECT 1 FROM nota n WHERE n.alumno_id = a.alumno_id
);
```

- A diferencia del JOIN, EXISTS **no duplica** filas aunque haya muchas coincidencias.

De muchas filas a un resumen

- Las **funciones de agregación** resumen muchas filas en **un** valor: promedios, totales, conteos.

Funciones de agregación

SELECT

COUNT(*) AS cantidad,

AVG(nota) AS promedio,

MIN(nota) AS minima,

MAX(nota) AS maxima,

SUM(nota) AS suma

FROM nota;

- Sin GROUP BY, agregan **toda la tabla** en una sola fila de resumen.

Las tres caras de COUNT

```
SELECT
    COUNT(*)                AS filas_totales,
    COUNT(nota)             AS notas_no_nulas,
    COUNT(DISTINCT alumno_id) AS alumnos_distintos
FROM nota;
```

- COUNT(*): cuenta **filas**.
- COUNT(columna): cuenta filas donde esa columna **no es NULL**.
- COUNT(DISTINCT columna): cuenta **valores distintos**.

GROUP BY: un resumen por grupo

```
-- Promedio de notas por curso
SELECT curso_id, AVG(nota) AS promedio
FROM nota
GROUP BY curso_id;
```

- GROUP BY divide las filas en grupos según el valor de las columnas indicadas.
- La función de agregación se calcula **dentro de cada grupo**.

GROUP BY por varias columnas

```
-- Promedio por curso y tipo de evaluación
SELECT curso_id, evaluacion, AVG(nota) AS promedio
FROM nota
GROUP BY curso_id, evaluacion
ORDER BY curso_id, evaluacion;
```

- El grupo es la **combinación** de valores: un grupo por cada par curso_id + evaluacion.

La regla de oro del GROUP BY

- En el SELECT, cada columna debe ser:
 - una columna que está en el GROUP BY, o
 - estar dentro de una función de agregación.

-- ERROR: 'evaluacion' no está agrupada ni agregada

```
SELECT curso_id, evaluacion, AVG(nota)
FROM nota
GROUP BY curso_id;
```

- La base de datos no sabría **qué** evaluacion mostrar por grupo, por eso falla.

HAVING: filtrar GRUPOS

```
-- Cursos cuyo promedio supera 5.0
SELECT curso_id, AVG(nota) AS promedio
FROM nota
GROUP BY curso_id
HAVING AVG(nota) > 5.0;
```

- WHERE filtra **filas**, antes de agrupar.
- HAVING filtra **grupos** después de agrupar, y puede usar agregados.

WHERE y HAVING juntos

```
-- Promedio de EXÁMENES por curso, solo donde supere 5.0
SELECT curso_id, AVG(nota) AS promedio_examen
FROM nota
WHERE evaluacion = 'Examen'      -- filtra filas antes de agrupar
GROUP BY curso_id
HAVING AVG(nota) > 5.0;          -- filtra grupos después
```

- Regla práctica: condición sobre **columnas crudas** → WHERE; condición sobre **agregados** → HAVING.

GROUP BY con JOIN: lo más común en la práctica

```
-- Promedio por NOMBRE de curso (no por id)
SELECT c.nombre AS curso, AVG(n.nota) AS promedio
FROM nota n
JOIN curso c ON c.curso_id = n.curso_id
GROUP BY c.nombre
ORDER BY promedio DESC;
```

- Primero se unen las tablas, luego se agrupa el resultado combinado.

Una consulta dentro de otra

- A veces necesitas un valor o una lista que **proviene de otra consulta**.
- Una subconsulta es un `SELECT` entre paréntesis dentro de otro `SELECT`.

```
-- Notas que están por encima del promedio general
SELECT evaluacion, nota
FROM nota
WHERE nota > (SELECT AVG(nota) FROM nota);
```

- La subconsulta (SELECT AVG(nota) FROM nota) devuelve **un número**.
- Se puede usar en cualquier lugar donde iría un valor.

Subconsulta en WHERE con IN

```
-- Notas de cursos dictados por el profesor 1
SELECT *
FROM nota
WHERE curso_id IN (
    SELECT curso_id FROM curso WHERE profesor_id = 1
);
```

- La subconsulta devuelve una **lista** de ids; IN comprueba pertenencia.

Subconsulta correlacionada

```
-- Notas por encima del promedio DE SU PROPIO curso
SELECT n.evaluacion, n.curso_id, n.nota
FROM nota n
WHERE n.nota > (
    SELECT AVG(n2.nota)
    FROM nota n2
    WHERE n2.curso_id = n.curso_id      -- depende de la fila externa
);
```

- “Correlacionada” = la subconsulta **referencia** la fila externa con n.curso_id.
- Se evalúa una vez por cada fila externa.

EXISTS y NOT EXISTS

-- Cursos que tienen al menos una nota

```
SELECT c.nombre  
FROM curso c  
WHERE EXISTS (SELECT 1 FROM nota n WHERE n.curso_id = c.curso_id);
```

-- Cursos sin ninguna nota

```
SELECT c.nombre  
FROM curso c  
WHERE NOT EXISTS (SELECT 1 FROM nota n WHERE n.curso_id = c.curso_id);
```

- EXISTS solo pregunta **si hay o no** filas. Da igual qué columnas proyectes dentro; SELECT 1 es solo una convención.

Subconsulta en FROM: tabla derivada

```
-- Promedio por curso, y luego me quedo con los buenos
SELECT *
FROM (
    SELECT curso_id, AVG(nota) AS promedio
    FROM nota
    GROUP BY curso_id
) AS resumen
WHERE resumen.promedio > 5.0;
```

- La subconsulta actúa como una **tabla temporal**; debe llevar un alias, como AS resumen.

Subconsultas con nombre y legibles

- Una **CTE**, o Common Table Expression, es una subconsulta a la que le pones nombre **antes** de usarla.
- Convierte consultas anidadas difíciles de leer en pasos claros.

WITH: el mismo ejemplo, más legible

```
WITH resumen AS (  
    SELECT curso_id, AVG(nota) AS promedio  
    FROM nota  
    GROUP BY curso_id  
)  
SELECT *  
FROM resumen  
WHERE promedio > 5.0;
```

- Se lee de arriba hacia abajo: “primero calculo resumen, después lo consulto”.

Varias CTEs encadenadas

```
WITH prom_curso AS (  
    SELECT curso_id, AVG(nota) AS promedio  
    FROM nota  
    GROUP BY curso_id  
)  
con_nombre AS (  
    SELECT c.nombre, p.promedio  
    FROM prom_curso p  
    JOIN curso c ON c.curso_id = p.curso_id  
)
```

- Una CTE puede **usar** a las anteriores: con_nombre se apoya en prom_curso.

CTEs: la consulta final

```
-- con_nombre(nombre, promedio) viene de las CTEs anteriores
SELECT *
FROM con_nombre
ORDER BY promedio DESC;
```

- La consulta final lee la última CTE como si fuera una tabla.

- Hacen lo mismo; la diferencia es **legibilidad**.
- Usa CTE cuando:
 - la consulta tiene varios pasos,
 - quieres **reutilizar** un resultado intermedio,
 - la subconsulta anidada se vuelve difícil de seguir.
- Para algo cortito, una subconsulta inline está bien.

Mención: CTE recursiva

```
-- Contar del 1 al 5 (ejemplo mínimo de recursión)
WITH RECURSIVE numeros AS (
    SELECT 1 AS n                                -- caso base
    UNION ALL
    SELECT n + 1 FROM numeros WHERE n < 5      -- paso recursivo
)
SELECT n FROM numeros;
```

- WITH RECURSIVE resuelve jerarquías como árboles o grafos: organigramas, categorías anidadas, etc. Tema avanzado, queda solo presentado.

Agregar SIN colapsar las filas

- Problema: quiero el promedio del curso **al lado de cada nota**, sin perder las filas individuales.
- GROUP BY colapsa; las **funciones ventana** calculan sobre un grupo pero **mantienen** todas las filas.

OVER: la clave de las funciones ventana

```
SELECT
    alumno_id,
    curso_id,
    nota,
    AVG(nota) OVER (PARTITION BY curso_id) AS promedio_curso
FROM nota;
```

- OVER (...) define la **ventana**: el conjunto de filas sobre el que se calcula.
- PARTITION BY curso_id = “una ventana por cada curso”.
- Cada fila conserva su nota **y** ve el promedio de su curso.

Numerar y rankear filas

SELECT

```
curso_id,  
alumno_id,  
nota,  
ROW_NUMBER() OVER w AS posicion,  
RANK()        OVER w AS rank,  
DENSE_RANK()  OVER w AS dense_rank
```

FROM nota

WINDOW w **AS** (**PARTITION BY** curso_id **ORDER BY** nota **DESC**);

- ROW_NUMBER: 1, 2, 3, 4... siempre único.
- RANK: empates comparten número y **saltan**: 1, 1, 3...
- DENSE_RANK: empates comparten número y **no saltan**: 1, 1, 2...
- WINDOW w AS (...) define la ventana una vez y la reusas con OVER w.

Mirar la fila anterior o siguiente: LAG / LEAD

```
-- Cómo cambió la nota respecto de la evaluación anterior
SELECT
    alumno_id, curso_id, evaluacion, fecha, nota,
    LAG(nota) OVER w AS nota_previa,
    nota - LAG(nota) OVER w AS variacion
FROM nota
WINDOW w AS (PARTITION BY alumno_id, curso_id ORDER BY fecha);
```

- LAG mira la fila **anterior** de la ventana; LEAD, la **siguiente**.
- Ideal para comparar contra el período previo.

Acumulados: running total

-- Promedio acumulado de un alumno a lo largo del tiempo

SELECT

alumno_id, curso_id, fecha, nota,

AVG(nota) **OVER** (

PARTITION BY alumno_id, curso_id

ORDER BY fecha

) **AS** promedio_hasta_la_fecha

FROM nota;

- Al agregar **ORDER BY** dentro del **OVER**, el cálculo es **acumulativo**: usa desde la primera fila hasta la actual.

El marco de la ventana: el frame

```
SELECT
    alumno_id, fecha, nota,
    SUM(nota) OVER (
        PARTITION BY alumno_id
        ORDER BY fecha
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS suma_acumulada
FROM nota;
```

- El **frame** define qué filas entran: aquí, desde el inicio hasta la actual.
- Es lo que está implícito cuando pones ORDER BY en una ventana de agregación.

Combinar resultados de varias consultas

- Combinar tablas con atributos diferentes requería JOIN.
- Existen operadores de conjuntos combinan **filas** de dos consultas que tienen las **mismas columnas**.
- Uno convina de forma *horizontal* mientras que el otro *vertical*.

UNION y UNION ALL

-- Nombres de alumnos y de profesores en una sola lista

```
SELECT nombre FROM alumno
```

```
UNION
```

```
SELECT nombre FROM profesor;
```

- UNION une y **elimina duplicados**.
- UNION ALL une y **conserva** duplicados; es más rápido porque no deduplica.
- Requisito: misma cantidad de columnas y tipos compatibles.

INTERSECT y EXCEPT

-- Cursos que tienen notas Y horario asignado

```
SELECT curso_id FROM nota
INTERSECT
SELECT curso_id FROM horario;
```

-- Cursos con notas pero SIN horario

```
SELECT curso_id FROM nota
EXCEPT
SELECT curso_id FROM horario;
```

- INTERSECT: filas que están en **ambas** consultas.
- EXCEPT: filas de la primera que **no** están en la segunda.

Las funciones que aparecen una y otra vez

- PostgreSQL trae cientos de funciones; estas son las que usarás a diario para transformar y presentar datos.

```
SELECT
    nombre,
    UPPER(nombre)          AS mayus,
    LOWER(nombre)          AS minus,
    LENGTH(nombre)         AS largo,
    SUBSTRING(nombre, 1, 3) AS inicial,
    'Alumno: ' || nombre   AS etiqueta  -- // concatena
FROM alumno;
```

- || concatena texto. También existen TRIM, REPLACE, POSITION.


```
SELECT
    nota,
    ROUND(nota)           AS redondeada,
    ROUND(nota, 0)        AS sin_decimales,
    CEIL(nota)            AS hacia_arriba,
    FLOOR(nota)           AS hacia_abajo,
    ABS(nota - 4.0)       AS distancia_al_4
FROM nota;
```

- ROUND, CEIL, FLOOR, ABS, MOD cubren la mayoría de los casos.

Funciones de fecha y hora

```
SELECT
```

```
    fecha,
```

```
    CURRENT_DATE           AS hoy,
```

```
    EXTRACT(MONTH FROM fecha) AS mes,
```

```
    DATE_TRUNC('month', fecha) AS inicio_de_mes,
```

```
    AGE(CURRENT_DATE, fecha)  AS antigüedad
```

```
FROM nota;
```

- EXTRACT saca una parte: año, mes, día.
- DATE_TRUNC “recorta” a una unidad, útil para agrupar por mes.
- Las fechas admiten aritmética: fecha + INTERVAL '7 days'.

Condicionales: CASE WHEN

```
SELECT
    evaluacion,
    nota,
    CASE
        WHEN nota >= 6.0 THEN 'Destacado'
        WHEN nota >= 4.0 THEN 'Aprobado'
        ELSE 'Reprobado'
    END AS estado
FROM nota;
```

- CASE es el “if/else” de SQL: evalúa condiciones en orden y devuelve la primera que se cumple.

Manejar NULL: COALESCE y NULLIF

-- Si la nota es NULL, mostrar 0

```
SELECT evaluacion, COALESCE(nota, 0) AS nota_o_cero  
FROM nota;
```

-- Convertir un valor "centinela" en NULL

```
SELECT NULLIF(nota, 0) FROM nota; -- 0 pasa a NULL
```

- COALESCE(a, b, ...): devuelve el **primer** valor no nulo.
- NULLIF(a, b): devuelve NULL si a = b, si no devuelve a.

Conversión de tipos: casting

SELECT

```
nota::TEXT           AS nota_texto,    -- sintaxis PostgreSQL
CAST(nota AS INTEGER) AS nota_entera,  -- sintaxis estándar SQL
'2025-06-09'::DATE   AS una_fecha
```

FROM nota;

- `valor::tipo` es el atajo de PostgreSQL; `CAST(valor AS tipo)` es el estándar.

Parte 10: cómo se ejecuta la consulta

Del plan lógico al plan real

- Ya vimos el orden **lógico** de ejecución: FROM → WHERE → GROUP BY → ...
- Pero el motor arma el plan **físico**: con qué algoritmo lee cada tabla, en qué orden las une y si usa índices o no.
- EXPLAIN te deja **ver ese plan** antes de confiar en una consulta lenta.

EXPLAIN: el plan, SIN ejecutar la consulta

-- Muestra el plan que la base USARÍA; NO ejecuta la consulta

EXPLAIN

SELECT * FROM nota **WHERE** nota >= 6.0;

- EXPLAIN solo **planifica y muestra**: no lee los datos ni mide tiempo real.
- Los números que entrega son **estimaciones** del planificador, no mediciones.
- Es seguro: aunque la consulta fuera un UPDATE, con EXPLAIN a secas **no** se toca nada.

-- Verás algo parecido a esto (tus números variarán):

```
Seq Scan on nota (cost=0.00..1.71 rows=22 width=30)
```

```
  Filter: (nota >= 6.0)
```

- `cost=arranque..total`: costo **estimado** en una unidad interna del planificador.
- `rows`: cuántas filas **estima** que devuelve ese nodo.
- `width`: ancho estimado de cada fila en bytes.
- `Filter`: la condición que el nodo aplica a las filas.

-- Plan de un JOIN nota-curso:

```
Hash Join (cost=1.58..3.33 rows=57)          <- 3. une ambos
  Hash Cond: (n.curso_id = c.curso_id)
    -> Seq Scan on nota n (cost=0.00..1.57)    <- 1. lee nota
    -> Hash (cost=1.26..1.26 rows=26)         <- 2. hashea curso
        -> Seq Scan on curso c (cost=0.00..1.26)
```

- Cada nodo entrega sus filas al nodo de **arriba**.
- La indentación marca quién alimenta a quién.

EXPLAIN ANALYZE: el plan REAL, ejecutándolo

-- Ahora SÍ ejecuta la consulta y mide lo que pasó de verdad

EXPLAIN ANALYZE

SELECT * FROM nota **WHERE** nota >= 6.0;

- Agrega a cada nodo el actual time y las rows **reales**, más el Execution Time total.
- **Cuidado:** ejecuta de verdad. Sobre un UPDATE/DELETE **modifica los datos**. Para probar sin efectos, envuélvelo en una transacción y haz ROLLBACK.

Estimado vs real: para qué sirve ANALYZE

```
Seq Scan on nota (cost=... rows=22 ...) (actual ... rows=21.00 ...)
           ^estimado                               ^real
```

Rows Removed by Filter: 36

- A la izquierda lo que el planificador **estimó** (rows=22); a la derecha lo que **ocurrió** (rows=21). Acá casi calzan: estadísticas sanas.
- Si estimado y real se separan **mucho**, el planificador está mal informado: corre ANALYZE nota; para actualizar las estadísticas.
- Ojo, son cosas distintas: ANALYZE (comando) recolecta estadísticas; EXPLAIN ANALYZE ejecuta y mide una consulta.

¿Qué es un índice?

- Un **índice** es una estructura auxiliar que deja a la base **encontrar filas por el valor de una columna** sin leer toda la tabla, como el índice de un libro.
- **No** es una copia de los datos ni algo que se use siempre: es una estructura aparte que apunta a las filas.
- PostgreSQL crea uno solo para cada PRIMARY KEY y UNIQUE; el resto los creas tú:

```
-- En nuestro modelo ya existe este índice sobre nota.curso_id  
CREATE INDEX idx_nota__curso ON nota (curso_id);
```

- Acelera las búsquedas, pero **ocupa espacio** y hay que mantenerlo en cada INSERT/UPDATE: no se crean “por si acaso”.

Seq Scan vs Index Scan

```
-- Existe un índice en nota.curso_id (idx_nota__curso)
```

```
EXPLAIN SELECT * FROM nota WHERE curso_id = 1;
```

Seq Scan on nota (cost=0.00..1.71 rows=6 width=30)

Filter: (curso_id = 1)

- Seq Scan: lee la tabla **completa**, fila por fila.
- Index Scan: usa el índice para ir **directo** a las filas que sirven.
- Aun habiendo índice, eligió Seq Scan: tener un índice **no** obliga a usarlo. ¿Por qué lo descartó?

¿Por qué Seq Scan si hay índice? Tablas chicas

- En tablas chicas como las de esta clase, casi siempre verás Seq Scan **aunque exista el índice**. No es un error.
- Leer ~60 filas de corrido es más barato que abrir el índice e ir fila por fila; el índice recién rinde con tablas **grandes** que devuelven **pocas** filas.

-- Forzar el índice, solo para DEMOSTRAR (no usar en producción):

```
SET enable_seqscan = off;
```

```
EXPLAIN SELECT * FROM nota WHERE curso_id = 1;
```

```
Index Scan using idx_nota__curso on nota (cost=0.14..8.25 rows=6)
```

```
Index Cond: (curso_id = 1)
```

- Estimó **8.25** para el índice vs **1.71** del Seq Scan: por eso no lo eligió.

Para qué te sirve EXPLAIN

- Confirmar si una consulta usa el índice que esperabas.
- Detectar un Seq Scan sobre una tabla **grande**: señal de índice faltante.
- Ver dónde se va el tiempo en una consulta lenta (con ANALYZE).
- Comparar dos formas de escribir la **misma** consulta.
*Primero haz que la consulta sea **correcta**; recién entonces usa EXPLAIN para hacerla **rápida**.*

Parte 11: recapitulación

```
SELECT  columnas, agregados, ventanas    -- 5. qué muestro
FROM    tabla                            -- 1. de dónde
JOIN    otra ON ...                      -- 1. con qué combino
WHERE    filtro_de_filas                 -- 2. qué filas
GROUP BY columnas                        -- 3. cómo agrupo
HAVING   filtro_de_grupos                -- 4. qué grupos
ORDER BY columnas                        -- 6. cómo ordeno
LIMIT n OFFSET m;                       -- 7. cuántas
```

- Los números son el **orden de ejecución**. Lo escribes arriba, se ejecuta en ese orden.

Receta para armar cualquier consulta

1. **¿De dónde salen los datos?** → FROM + los JOIN necesarios.
2. **¿Qué filas me sirven?** → WHERE.
3. **¿Necesito resumir?** → GROUP BY + agregados; filtra grupos con HAVING.
4. **¿Necesito un valor de otra consulta?** → subconsulta o CTE.
5. **¿Quiero un cálculo por fila sin perder detalle?** → función ventana.
6. **¿Cómo lo presento?** → SELECT, ORDER BY, LIMIT.

Ejemplo final: todo junto (1/2)

```
-- Top 1 alumno por curso, solo en cursos con promedio sobre 5.0
WITH ranking AS (
    SELECT
        c.nombre AS curso, a.nombre AS alumno,
        AVG(n.nota) AS promedio_alumno,
        RANK() OVER (
            PARTITION BY n.curso_id
            ORDER BY AVG(n.nota) DESC
        ) AS posicion
    FROM nota n
    JOIN alumno a ON a.alumno_id = n.alumno_id
    JOIN curso c ON c.curso_id = n.curso_id
    GROUP BY c.nombre, a.nombre, n.curso_id
) (...)
```

Ejemplo final: todo junto (2/2)

```
-- Top 1 alumno por curso, solo en cursos con promedio sobre 5.0  
(...)  
SELECT curso, alumno, ROUND(promedio_alumno, 2) AS promedio  
FROM ranking  
WHERE posicion = 1  
ORDER BY promedio DESC;
```

Lo que combinamos en ese ejemplo

- **JOIN** de tres tablas: nota, alumno, curso.
- **GROUP BY** + **AVG** para el promedio de cada alumno por curso.
- **Función ventana RANK** para ordenar dentro de cada curso.
- **CTE** para calcular el ranking y luego filtrarlo.
- **Filtro, redondeo y orden** para presentar el resultado.

Ninguna pieza es nueva: solo aplicamos el mapa.

Lo que te llevas

- Toda consulta es el mismo esqueleto; tú vas rellenando piezas.
- El **orden de ejecución** explica qué se puede usar dónde.
- WHERE filtra filas, HAVING filtra grupos.
- CTEs y subconsultas dan pasos intermedios; las funciones ventana resumen sin colapsar.
- La mejor forma de fijarlo es **escribir consultas**.

¿Tienes dudas? ¡Hablemos!